

Развёртывание системы BHS.CRG

Система исполнительной документации BHS.CRG

Версия документа: 2026-06-29

Инструкция по развёртыванию BHS.CRG

Система генерации исполнительной документации для электромонтажных проектов. Развёртывание — через **Docker Compose** на сервере **Linux**: весь стек (веб-интерфейс, сервер приложений, база данных, объектное хранилище, модель распознавания) поднимается несколькими командами.

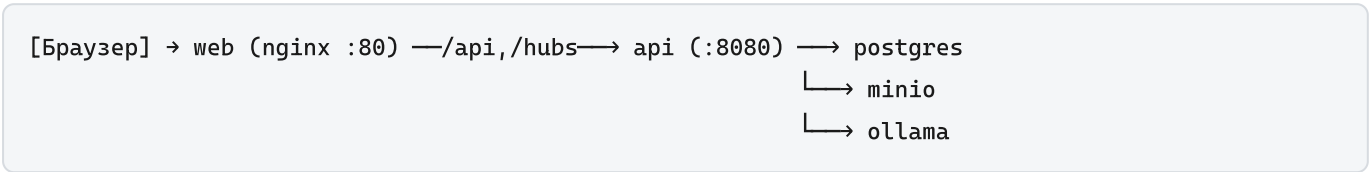
Этот документ — для администратора/инженера, разворачивающего систему. Для работы с системой см. «Инструкцию пользователя» и «Инструкцию администратора».

1. Архитектура развёртывания

Компонент	Образ / технология	Назначение
web	nginx + собранный React-SPA	Веб-интерфейс; проксирует /api и /hubs на сервер
api	ASP.NET Core 10 + Typst	Сервер приложений, REST API, генерация PDF
postgres	PostgreSQL 16	Основная база данных
minio	MinIO	Объектное хранилище (сканы, наборы данных, готовые файлы)
ollama	Ollama	Локальная модель распознавания сканов (опционально)

Вспомогательные одноразовые сервисы: `createbuckets` (создаёт bucket в MinIO), `ollama-init` (загружает модель распознавания).

Схематично:



Генерация: PDF собирается движком **Typst** (входит в образ `api`). Формат **DOCX в текущей версии не поддерживается** — только PDF.

2. Требования

- **ОС:** Linux (Ubuntu 22.04+/Debian 12+ и совместимые).
- **Docker Engine 24+** и **Docker Compose v2** (`docker compose`, не `docker-compose`).
- **Ресурсы:**
 - Базовый стек (без Ollama): от **2 CPU / 4 ГБ RAM / 10 ГБ диска**.
 - С Ollama и моделью `qwen2.5vl:7b`: дополнительно **~8 ГБ RAM** и **~6 ГБ диска** под модель; желательна видеокарта, но работает и на CPU (медленнее).
- Доступ в интернет на этапе сборки (загрузка образов, Typst, npm/NuGet-пакетов) и, при использовании веб-поиска документов качества, в рантайме.

Проверка окружения:

```
docker --version
docker compose version
```

3. Быстрый старт

```
# 1. Получить исходный код
git clone https://github.com/pavru/bhs.crg.git
cd bhs.crg

# 2. Подготовить конфигурацию
cp deploy/.env.example deploy/.env
nano deploy/.env          # задать пароли и JWT-секрет (см. раздел 4)

# 3. Собрать и запустить весь стек
docker compose -f deploy/docker-compose.yml up -d --build

# 4. Проверить статус
docker compose -f deploy/docker-compose.yml ps
docker compose -f deploy/docker-compose.yml logs -f api
```

После запуска веб-интерфейс доступен по адресу **http://СЕРВЕР:8080** (порт задаётся `WEB_PORT` в `.env`).

При первом старте сервер `api` автоматически применяет миграции базы данных и создаёт роли `Admin / User`. Отдельных команд миграции выполнять не нужно.

4. Конфигурация (`deploy/.env`)

Скопируйте `deploy/.env.example` → `deploy/.env` и заполните. Файл `.env` содержит секреты и **не** хранится в репозитории.

Переменная	Назначение	Обязательно
POSTGRES_DB / POSTGRES_USER / POSTGRES_PASSWORD	Параметры БД	да (сменить пароль)
MINIO_ROOT_USER / MINIO_ROOT_PASSWORD	Учётные данные MinIO	да (сменить)
MINIO_BUCKET	Имя bucket для файлов	да (по умолч. bhs-crg)
MINIO_CONSOLE_PORT	Порт веб-консоли MinIO на хосте	нет
JWT_KEY	Секрет подписи токенов, ≥ 32 СИМВОЛОВ	да
WEB_PORT	Порт веб-интерфейса на хосте	нет (по умолч. 8080)
OLLAMA_MODEL	Модель распознавания	нет (по умолч. qwen2.5vl:7b)
ANTHROPIC_API_KEY	Распознавание реквизитов (Claude)	опц.
GEMINI_API_KEY	Распознавание реквизитов (Gemini)	опц.
SERPER_API_KEY	Веб-поиск документов качества	опц.

Сгенерировать надёжный JWT_KEY :

```
openssl rand -base64 48
```

Важно: обязательно смените пароли БД/MinIO и JWT_KEY перед вводом в эксплуатацию. Значения из примера — только для ознакомления.

5. Первый вход (создание администратора)

1. Откройте **http://SERVER:8080**.
2. Пока в системе нет ни одного пользователя, доступна **регистрация первого администратора**. Зарегистрируйте учётную запись — она автоматически получит роль **Администратор**.
3. После этого **публичная регистрация закрывается**. Все остальные пользователи создаются администратором в разделе **Настройка системы** → **Пользователи** (см. «Инструкцию администратора»).

6. Порты

Сервис	Внутренний порт	Порт на хосте	Примечание
web (nginx)	80	WEB_PORT (8080)	Основной вход в систему
api	8080	—	Доступен только внутри сети Compose
postgres	5432	—	Наружу не публикуется
minio (API)	9000	—	Наружу не публикуется
minio (консоль)	9001	MINIO_CONSOLE_PORT (9001)	Админ-консоль MinIO (по желанию)
ollama	11434	—	Наружу не публикуется

По умолчанию наружу выведены только веб-интерфейс и (опционально) консоль MinIO. Базу, хранилище и Ollama не следует публиковать в интернет.

7. Опциональные компоненты

Распознавание сканов (документы качества)

- **Ollama (локально):** модель загружается сервисом `ollama-init` при первом `up`. Загрузить/обновить вручную:

```
docker compose -f deploy/docker-compose.yml exec ollama ollama pull qwen2.5vl:7b
```

- **Облачные модели:** задайте `ANTHROPIC_API_KEY` и/или `GEMINI_API_KEY` в `.env`. Порядок перебора движков настраивается в коде (`Recognition:Order`) и в разделе настроек интеграций.

Веб-поиск документов качества

- Задайте `SERPER_API_KEY` (`serper.dev`) в `.env`. Без ключа поиск в интернете недоступен, остальная работа с документами качества — да.

Ключи интеграций можно также задавать/менять в самом приложении: **Настройка системы** → **Настройки** → **Поиск и распознавание** (значения хранятся в БД и маскируются при чтении).

8. Обновление системы

```
cd bhs.crg
git pull
docker compose -f deploy/docker-compose.yml up -d --build
```

Миграции БД применяются автоматически при старте `api`. Данные сохраняются в томах (`postgres_data`, `minio_data`, `ollama_data`).

9. Резервное копирование и восстановление

База данных:

```
# Бэкап
docker compose -f deploy/docker-compose.yml exec -T postgres \
  pg_dump -U postgres bhs_crg > backup_$(date +%F).sql

# Восстановление
cat backup_YYYY-MM-DD.sql | docker compose -f deploy/docker-compose.yml exec -T postgres \
  psql -U postgres -d bhs_crg
```

Файлы (MinIO): резервируйте том `minio_data` (или зеркалируйте bucket через `mc mirror`).

В приложении также есть встроенное **резервное копирование** для администратора (**Настройка системы** → **Настройки** → **Резервное копирование**).

10. Диагностика

Симптом	Причина / решение
<code>api</code> падает при старте	Проверьте <code>JWT_KEY</code> (≥ 32 симв.) и доступность <code>postgres</code> (healthcheck). <code>docker compose logs api</code>
Генерация PDF с ошибкой шрифтов	В образ включены DejaVu/Liberation/Noto. Если шаблон требует особый шрифт — добавьте его в <code>Dockerfile.api</code>
Распознавание не работает	Модель Ollama не загружена (<code>ollama pull</code>) или не задан API-ключ. Это опциональная функция
Веб-интерфейс открывается, но «не видит» сервер	Проверьте, что сервис <code>web</code> проксирует на <code>api:8080</code> (см. <code>deploy/nginx.conf</code>)
Не загружаются крупные файлы	Увеличьте <code>client_max_body_size</code> в <code>deploy/nginx.conf</code>

Полезные команды:

```
docker compose -f deploy/docker-compose.yml ps          # статус
docker compose -f deploy/docker-compose.yml logs -f api  # логи сервера
docker compose -f deploy/docker-compose.yml down         # остановить (тома сохраняются)
docker compose -f deploy/docker-compose.yml down -v      # остановить и УДАЛИТЬ данные
```

11. Безопасность (чек-лист перед эксплуатацией)

- ☐ Сменены пароли `POSTGRES_PASSWORD` , `MINIO_ROOT_PASSWORD` .
- ☐ Задан длинный случайный `JWT_KEY` (≥ 32 символов).

- ☐ Порты БД/MinIO/Ollama не опубликованы в интернет.
- ☐ Перед публикацией наружу настроен HTTPS (обратный прокси/терминатор TLS перед web).
- ☐ Создан администратор; лишние тестовые учётные записи удалены.
- ☐ Настроено регулярное резервное копирование БД и minio_data .